

PesaGuard

Real-Time Reconciliation & Anomaly Detection for M-Pesa-Integrated Businesses

Version	MVP 0.1
Prepared for	Victor Kipruto Rop · DataForge
Last updated	July 2026
Scope	Architecture, API reference, data models, deployment

This document accompanies the PesaGuard MVP scaffold and Project Blueprint. It is the working technical reference for engineers building on the codebase.

CONTENTS

Table of Contents

01	Overview
02	Architecture
03	Project Structure
04	Getting Started
05	Environment Variables
06	API Reference
07	Data Models
08	Kafka Topics
09	Anomaly Detection Rules
10	Internal-System Connectors
11	Alerting
12	Testing
13	Deployment
14	Troubleshooting
15	Roadmap

01 Overview

PesaGuard ingests M-Pesa Daraja webhook callbacks, streams them through Kafka, reconciles them against a business's internal ledger/orders, and alerts on discrepancies — duplicate transactions, amount mismatches, missing callbacks, and unusually large payments — in near real-time.

Core Services

Service	Location	Responsibility
Webhook Receiver	ingestion/webhook_receiver/	Validates and ingests Daraja callbacks
Reconciliation Job	streaming/flink_jobs/	Consumes Kafka stream, applies anomaly rules
Dashboard API	dashboard/api/	Serves discrepancy/summary data
Alerting	alerting/	Dispatches Slack notifications on discrepancies

02 Architecture

M-Pesa Daraja sends webhook callbacks to the Flask receiver, which validates the payload and publishes it to a Kafka topic. The reconciliation service consumes this stream, pulls matching records from the tenant's internal system via a pluggable connector, applies matching and anomaly rules, and writes results to Postgres. Discrepancies trigger alerts; the dashboard reads from Postgres to show live status.

```
M-Pesa Daraja
|
v
Webhook Receiver (Flask)
| validates + publishes
v
Kafka: mpesa.transactions.raw
|
v
Reconciliation Job <----> Internal Ledger Connector
|
| writes
v
PostgreSQL (transactions, discrepancies)
|
+-----+
v               v
Alerting         Dashboard API
(Slack / SMS / email) (Flask)
```

Design Principles

- Kafka decouples ingestion from processing — the webhook receiver never blocks on reconciliation logic, so a slow rule engine can't cause Daraja callback timeouts.

- Postgres is the single source of truth for audit purposes — every transaction and discrepancy decision is persisted, not just the flagged ones.
- Connectors are pluggable — the reconciliation job doesn't know or care whether a tenant's internal data lives in Postgres, Sheets, or a REST API.

03 Project Structure

```
pesaguard/
├── ingestion/
│   ├── webhook_receiver/           Flask app: receives Daraja callbacks
│   │   ├── app.py                 Routes: /webhook/mpesa/confirmation, /validation
│   │   ├── validators.py          Payload validation
│   │   ├── producer.py            Kafka publish wrapper
│   │   └── Dockerfile
│   ├── internal_sync/connectors/
│   │   └── base_connector.py       Abstract connector interface
│   └── streaming/
│       ├── kafka/topics.py         Topic name constants
│       ├── flink_jobs/
│       │   ├── reconciliation_job.py Main consumer loop
│       │   ├── anomaly_rules.py     Rule definitions
│       │   └── Dockerfile
│       └── storage/
│           ├── models/models.py     SQLAlchemy models
│           └── migrations/init_db.py Table creation script
│       └── alerting/
│           ├── notifier.py          Slack webhook dispatch
│           └── templates/slack_template.md
│       └── dashboard/api/
│           ├── app.py              Discrepancy/summary endpoints
│           └── Dockerfile
│       └── infra/docker-compose.yml  Local dev stack
│       └── tests/test_anomaly_rules.py
│       └── .env.example
│       └── README.md
```

04 Getting Started

Prerequisites

- Docker & Docker Compose
- Python 3.11+ (for running scripts outside containers)
- A Daraja sandbox account — developer.safaricom.co.ke

Setup

```
# 1. Clone/extract the project, then:
cp .env.example .env

# 2. Fill in .env with Daraja sandbox credentials + SLACK_WEBHOOK_URL
#    (optional for local dev -- logs instead if unset)

# 3. Start the stack
cd infra/
docker-compose up -d

# 4. Initialize the database
python storage/migrations/init_db.py

# 5. Verify services are healthy
curl http://localhost:5000/health
curl http://localhost:5001/api/stats/summary
```

Exposing Your Local Webhook Receiver to Daraja

Daraja needs a public URL to send callbacks to. For local development, use a tunnel such as ngrok, then register the tunnel URL plus `/webhook/mpesa/confirmation` as your ConfirmationURL in the Daraja sandbox portal.

```
ngrok http 5000
```

05 Environment Variables

Variable	Required	Default	Description
DARAJA_CONSUMER_KEY	Yes (prod)	—	Daraja API consumer key
DARAJA_CONSUMER_SECRET	Yes (prod)	—	Daraja API consumer secret
DARAJA_SHORTCODE	Yes (prod)	—	Business shortcode/till number
DARAJA_ENV	No	sandbox	sandbox or production
KAFKA_BOOTSTRAP_SERVERS	No	localhost:9092	Kafka broker address
KAFKA_TOPIC_TRANSACTIONS	No	mpesa.transactions.raw	Raw ingestion topic
KAFKA_TOPIC_MATCHED	No	mpesa.transactions.matched	Clean reconciliations
KAFKA_TOPIC_DISCREPANCIES	No	mpesa.discrepancies	Flagged anomalies
DATABASE_URL	No	postgresql://pesaguard:...	Postgres connection string
SLACK_WEBHOOK_URL	No	—	If unset, alerts are logged instead of sent
PORT	No	5000	Webhook receiver port
DASHBOARD_API_PORT	No	5001	Dashboard API port

06 API Reference

Webhook Receiver

POST /webhook/mpesa/confirmation

Receives C2B confirmation callbacks from Daraja after a payment completes.

```
// Request body (Daraja-defined shape)
{
  "TransactionType": "Pay Bill",
  "TransID": "0EI2AK4Q16",
  "TransTime": "20260702143000",
  "TransAmount": "500.00",
  "BusinessShortCode": "600000",
  "MSISDN": "254712345678"
}

// Response
{ "ResultCode": 0, "ResultDesc": "Accepted" }
```

Returns `ResultCode: 1` with a 400 status if the payload fails validation (missing required fields, non-numeric amount, empty `TransID`).

POST /webhook/mpesa/validation

Receives C2B validation callbacks (pre-confirmation). Currently accepts all transactions by default — add business rules in `app.py` to reject transactions before completion.

GET /health

Returns `{"status": "ok"}` — used for container/orchestration health checks.

Dashboard API

GET /api/discrepancies

Returns all unresolved discrepancies, most recent first.

```
[
  {
    "id": "0EI2AK4Q16-duplicate_transaction_id",
    "trans_id": "0EI2AK4Q16",
    "anomaly_type": "duplicate_transaction_id",
    "details": null,
    "detected_at": "2026-07-02T14:30:12+00:00"
  }
]
```

GET /api/stats/summary

Returns a high-level reconciliation snapshot.

```
{
  "total_transactions": 4213,
  "open_discrepancies": 7,
  "reconciliation_rate": 0.9983
}
```


07 Data Models

Defined in `storage/models/models.py` (SQLAlchemy).

Transaction

Column	Type	Notes
trans_id	String (PK)	M-Pesa transaction ID
trans_amount	Float	
msisdn	String	Payer phone number
business_short_code	String	
trans_time	String	Raw Daraja timestamp format
raw_payload	JSON	Full original callback payload
created_at	DateTime	

Discrepancy

Column	Type	Notes
id	String (PK)	{trans_id}-{rule_name}
trans_id	String	
anomaly_type	String	e.g. duplicate_transaction_id
details	Text	Free-form notes
resolved	Boolean	Default false
detected_at	DateTime	
resolved_at	DateTime	Nullable

InternalRecord

Column	Type	Notes
internal_ref	String (PK)	Order/invoice ID in customer's system
amount	Float	
phone_number	String	
status	String	e.g. pending, paid, failed

Column	Type	Notes
synced_at	DateTime	

08 Kafka Topics

Topic	Producer	Consumer	Purpose
mpesa.transactions.raw	Webhook Receiver	Reconciliation Job	Every validated incoming M-Pesa event
mpesa.transactions.matched	Reconciliation Job	(future: analytics)	Transactions that passed all checks cleanly
mpesa.discrepancies	Reconciliation Job	Alerting	Transactions that failed one or more anomaly rules

09 Anomaly Detection Rules

Defined in `streaming/flink_jobs/anomaly_rules.py`. Each rule is independent; a transaction can trigger multiple rules at once.

Rule	Trigger Condition	Configurable?
<code>duplicate_transaction_id</code>	TransID already seen in this run	No — in-memory set; use Redis/DB for production
<code>amount_exceeds_threshold_{N}_KES</code>	<code>TransAmount > LARGE_AMOUNT_THRESHOLD_KES</code> (default 150,000)	Yes — edit constant in <code>anomaly_rules.py</code>
<code>invalid_or_zero_amount</code>	<code>TransAmount</code> is ≤ 0 or non-numeric	No

Not Yet Implemented

- `missing_internal_record` — M-Pesa confirms payment, no matching internal order
- `amount_mismatch` — M-Pesa amount $\neq$ internal record amount
- `till_number_mismatch`

10 Internal-System Connectors

Defined in `ingestion/internal_sync/connectors/base_connector.py`. All connectors implement:

```
def fetch_recent_records(self, since_minutes: int = 15) -> Iterable[Dict]:
    """Returns normalized records:
    { "internal_ref": str, "amount": float, "phone_number": str,
      "timestamp": str (ISO8601), "status": str }
    """
```

Provided stubs (require implementation before use):

- `PostgresConnector` — queries a customer's orders table directly
- `GoogleSheetsConnector` — reads from a customer's tracking sheet

To add a new connector, subclass `BaseConnector` and implement `fetch_recent_records()`, returning records in the normalized shape above.

11 Alerting

`alerting/notifier.py::send_slack_alert()` posts to `SLACK_WEBHOOK_URL`. If unset, it logs the discrepancy instead of failing — safe for local dev without a real Slack workspace.

```
■ PesaGuard Discrepancy Detected
Transaction: {trans_id}
Issues: {anomaly_list}
Detected at: {timestamp}
```

To add SMS or email channels, add a new function alongside `send_slack_alert()` in `notifier.py` and call it from the reconciliation job wherever discrepancies are dispatched.

12 Testing

```
pip install pytest --break-system-packages
pytest tests/
```

Current coverage: anomaly rule logic (`test_anomaly_rules.py`) — clean transactions, duplicates, large amounts, zero/invalid amounts.

Not Yet Covered

- Webhook payload validation edge cases
- Kafka producer/consumer integration
- Connector implementations
- Dashboard API endpoints

13 Deployment

Local / Development

`docker-compose up -d` from `infra/` — starts Zookeeper, Kafka, Postgres, and all three application services.

Production (recommended path for early pilots)

Single VM running the same `docker-compose.yml`. Sufficient for one to a handful of pilot tenants. Migrate to Kubernetes only once tenant count or throughput genuinely requires it.

Before Going to Production

- Set `DARAJA_ENV=production` and use live Daraja credentials
- Set a real `SLACK_WEBHOOK_URL`
- Move `DATABASE_URL` to a managed Postgres instance (backups matter for financial data)
- Put the webhook receiver behind HTTPS (Daraja requires a public HTTPS ConfirmationURL)
- Review Kenya's Data Protection Act obligations for storing customer transaction data

14 Troubleshooting

Symptom	Likely Cause	Fix
Webhook receiver returns 400	Payload missing required Daraja fields	Check validators.py REQUIRED_FIELDS; confirm Daraja callback type
No events reaching Kafka	Kafka not reachable from webhook receiver	Check KAFKA_BOOTSTRAP_SERVERS; confirm container health
Reconciliation job not consuming	Consumer group offset issue or Kafka down	Check docker-compose logs kafka; verify topic exists
No Slack alerts arriving	SLACK_WEBHOOK_URL unset or invalid	Check logs -- alerts are logged locally if webhook URL missing
Dashboard API returns empty stats	init_db.py not run, or DATABASE_URL mismatch	Re-run migrations; confirm all services share the same .env
Duplicate alerts for same transaction	In-memory dedupe set reset on restart	Expected at MVP scale -- see Roadmap for persistent dedupe

15 Roadmap

Items intentionally deferred from the MVP, in rough priority order:

- **Real cross-system matching** — phone+amount+time-window matching against internal records
- **Persistent deduplication** — replace in-memory set with Redis or a DB unique constraint
- **Multi-tenancy** — scope all tables/topics/API routes by tenant_id
- **Connector implementations** — finish Postgres and Google Sheets connectors against a real pilot customer
- **Dashboard frontend** — Next.js UI consuming the existing API
- **SMS alerting** via Africa's Talking
- **Dead-letter queue** for events that fail processing
- **Structured JSON logging** with tenant_id/trans_id correlation

For the full phased build plan and detailed requirements, see *PesaGuard_Project_Blueprint.pdf*. For a ready-to-use prompt to accelerate this roadmap with an AI coding agent, see *PESAGUARD_BUILD_PROMPT.md*.